

Body Performance Classification

Contents

Read data	1
Statistical description of the dataset	1
Data preprocessing	2
Visualize cleaned data	9
Perform some tests	13
Model	22

Read data

```
data <- read.csv("bodyPerformance.csv")
data <- clean_names(data)
```

Statistical description of the dataset

Check the structure of the data

```
dim(data)
```

```
## [1] 13393    12
```

```
glimpse(data)
```

```
## Rows: 13,393
## Columns: 12
## $ age                <dbl> 27, 25, 31, 32, 28, 36, 42, 33, 54, 28, 42, 57~
## $ gender             <chr> "M", "M", "M", "M", "M", "F", "F", "M", "M", "~
## $ height_cm          <dbl> 172.3, 165.0, 179.6, 174.5, 173.8, 165.4, 164.~
## $ weight_kg          <dbl> 75.24, 55.80, 78.00, 71.10, 67.70, 55.40, 63.7~
## $ body_fat           <dbl> 21.3, 15.7, 20.1, 18.4, 17.1, 22.0, 32.2, 36.9~
## $ diastolic          <dbl> 80, 77, 92, 76, 70, 64, 72, 84, 85, 81, 63, 69~
## $ systolic           <dbl> 130, 126, 152, 147, 127, 119, 135, 137, 165, 1~
## $ grip_force         <dbl> 54.9, 36.4, 44.8, 41.4, 43.5, 23.8, 22.7, 45.9~
## $ sit_and_bend_forward_cm <dbl> 18.4, 16.3, 12.0, 15.2, 27.1, 21.0, 0.8, 12.3,~
## $ sit_ups_counts     <dbl> 60, 53, 49, 53, 45, 27, 18, 42, 34, 55, 68, 0,~
## $ broad_jump_cm      <dbl> 217, 229, 181, 219, 217, 153, 146, 234, 148, 2~
## $ class              <chr> "C", "A", "C", "B", "B", "B", "D", "B", "C", "~
```

Statistical description of the data

```
summary(data)
```

```
##      age      gender      height_cm      weight_kg
## Min.   :21.00   Length:13393   Min.    :125.0   Min.    : 26.30
## 1st Qu.:25.00   Class :character   1st Qu.:162.4   1st Qu.: 58.20
## Median :32.00   Mode  :character   Median :169.2   Median : 67.40
## Mean   :36.78                                Mean   :168.6   Mean    : 67.45
## 3rd Qu.:48.00                                3rd Qu.:174.8   3rd Qu.: 75.30
## Max.   :64.00                                Max.    :193.8   Max.    :138.10
##      body_fat      diastolic      systolic      grip_force
## Min.    : 3.00   Min.    : 0.0   Min.    : 0.0   Min.    : 0.00
## 1st Qu.:18.00   1st Qu.: 71.0   1st Qu.:120.0   1st Qu.:27.50
## Median :22.80   Median : 79.0   Median :130.0   Median :37.90
## Mean    :23.24   Mean    : 78.8   Mean    :130.2   Mean    :36.96
## 3rd Qu.:28.00   3rd Qu.: 86.0   3rd Qu.:141.0   3rd Qu.:45.20
## Max.    :78.40   Max.    :156.2   Max.    :201.0   Max.    :70.50
## sit_and_bend_forward_cm sit_ups_counts broad_jump_cm      class
## Min.    :-25.00   Min.    : 0.00   Min.    : 0.0   Length:13393
## 1st Qu.: 10.90   1st Qu.:30.00   1st Qu.:162.0   Class :character
## Median : 16.20   Median :41.00   Median :193.0   Mode  :character
## Mean    : 15.21   Mean    :39.77   Mean    :190.1
## 3rd Qu.: 20.70   3rd Qu.:50.00   3rd Qu.:221.0
## Max.    :213.00   Max.    :80.00   Max.    :303.0
```

Define numerical and categorical features

```
numerical_features <- c("age", "height_cm", "weight_kg", "body_fat", "diastolic", "systolic",
                        "grip_force", "sit_and_bend_forward_cm", "sit_ups_counts",
                        "broad_jump_cm")

categorical_features <- c("gender", "class")
```

Data preprocessing

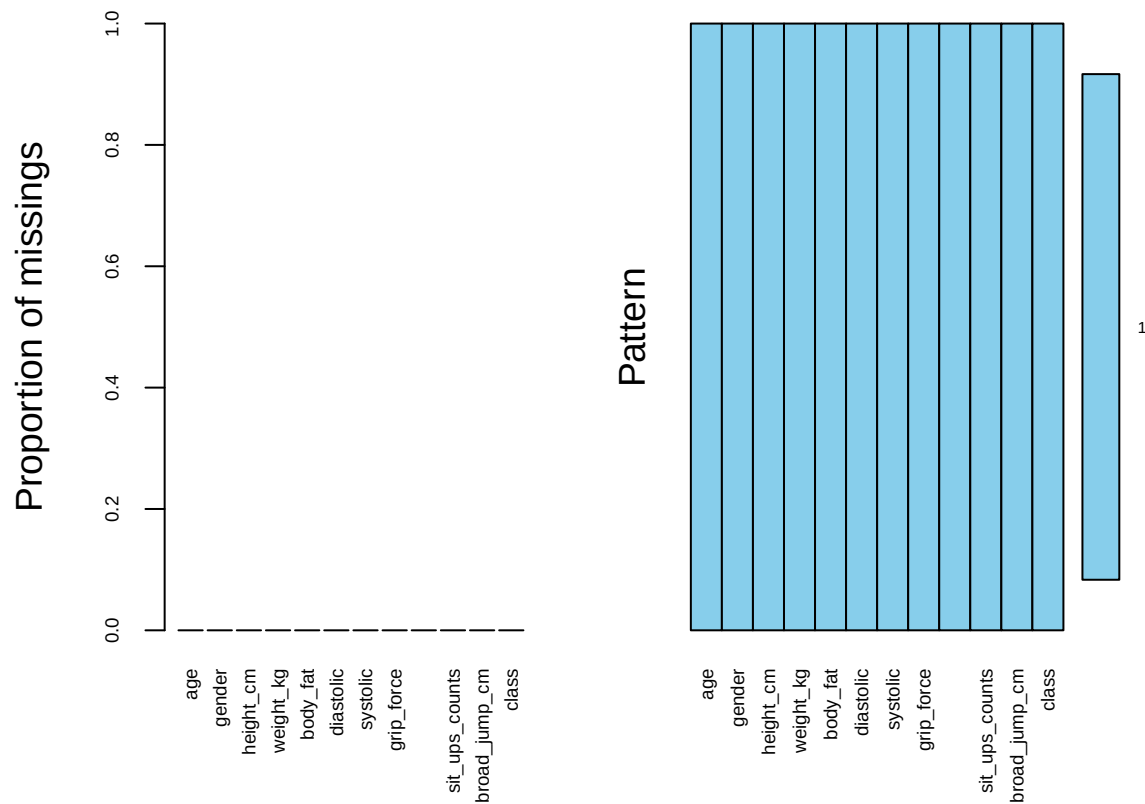
Check missing data

```
na_percentage <- colSums(is.na(data)) / nrow(data) * 100
na_percentage
```

```
##      age      gender      height_cm
##      0      0      0
## weight_kg      body_fat      diastolic
##      0      0      0
```

```
##          systolic          grip_force sit_and_bend_forward_cm
##              0              0              0
##    sit_ups_counts    broad_jump_cm          class
##              0              0              0
```

```
aggr(data, ylab = c("Proportion of missings", "Pattern"), number = TRUE,
      cex.axis = 0.6, cex.numbers = 0.5)
```



- There is no missing data.

Check duplicate data

```
sum(duplicated(data))
```

```
## [1] 1
```

```
data[duplicated(data), ]
```

```
##    age gender height_cm weight_kg body_fat diastolic systolic grip_force
## 12474  27      F    157     49.1    30.7       70       86      27.7
##    sit_and_bend_forward_cm sit_ups_counts broad_jump_cm class
## 12474                19.7           51          167      A
```

- There is 1 duplicate row.
- With only 1 duplicate row, the impact on the overall analysis may not be significant. However, keeping duplicate rows may lead to slight bias in descriptive statistics or modeling.

```
data <- data[!duplicated(data), ]

dim(data)
```

Remove duplicate rows

```
## [1] 13392    12
```

```
sum(duplicated(data))
```

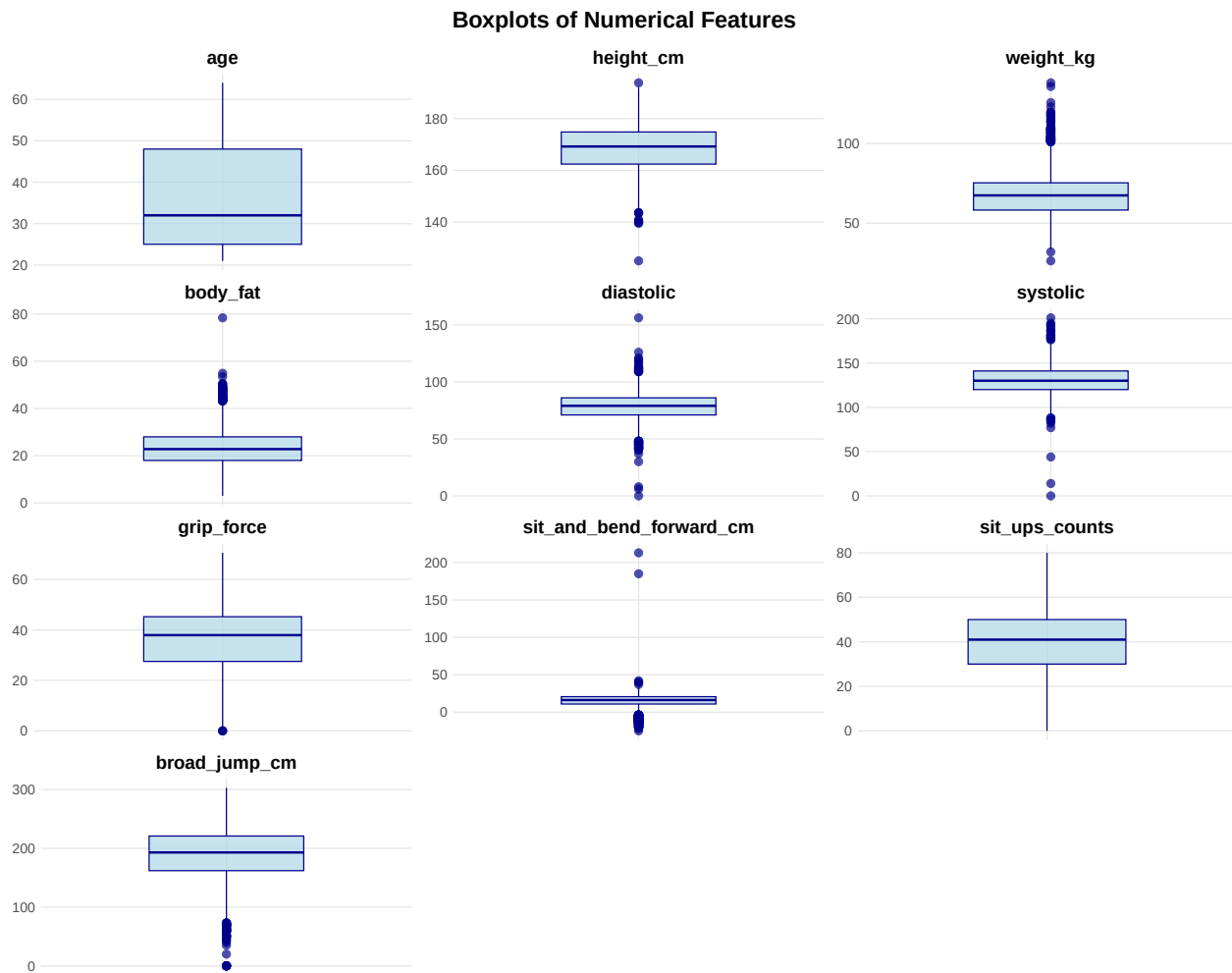
```
## [1] 0
```

Check outliers

```
boxplot_list <- list()

for (feature in numerical_features) {
  p <- ggplot(data, aes(x = "", y = .data[[feature]])) +
    geom_boxplot(fill = "lightblue", color = "darkblue", alpha = 0.7,
                 width = 0.5, outlier.size = 3) +
    labs(title = feature) +
    theme_minimal() +
    theme(
      plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
      panel.grid.major = element_line(color = "grey90"),
      panel.grid.minor = element_blank(),
      axis.title = element_blank(),
      axis.text.x = element_blank(),
      axis.text.y = element_text(size = 12)
    )
  boxplot_list[[feature]] <- p
}

grid.arrange(
  grobs = boxplot_list,
  ncol = 3,
  nrow = 4,
  top = textGrob(
    "Boxplots of Numerical Features",
    gp = gpar(fontsize = 20, fontface = "bold")
  ),
  padding = unit(2, "line")
)
```



- Quick note:
 - Most variables have outliers
 - Special variables diastolic, systolic, body_fat, sit_and_bend_forward_cm have outliers that are unreasonable with the real world
 - Consider removing outliers or replacing them

```
# Remove outliers based on IQR
remove_outliers <- function(column) {
  Q1 <- quantile(column, 0.25, na.rm = TRUE)
  Q3 <- quantile(column, 0.75, na.rm = TRUE)
  IQR <- Q3 - Q1
  lower_bound <- Q1 - 1.5 * IQR
  upper_bound <- Q3 + 1.5 * IQR
  column >= lower_bound & column <= upper_bound
}

# Winsorization function
winsorize <- function(column) {
  Q1 <- quantile(column, 0.25, na.rm = TRUE)
```

```

Q3 <- quantile(column, 0.75, na.rm = TRUE)
IQR <- Q3 - Q1
lower_bound <- Q1 - 1.5 * IQR
upper_bound <- Q3 + 1.5 * IQR
column[column < lower_bound] <- lower_bound
column[column > upper_bound] <- upper_bound
return(column)
}

# Start processing data
data <- data %>%
  # Remove outliers of diastolic and systolic
  filter(remove_outliers(diastolic), remove_outliers(systolic)) %>%
  # Remove negative values and outliers of sit_and_bend_forward_cm
  filter(sit_and_bend_forward_cm >= 0) %>%
  filter(remove_outliers(sit_and_bend_forward_cm)) %>%
  # BWinsorization cho body_fat
  mutate(body_fat = winsorize(body_fat))

summary(data)

```

Handle outliers

```

##      age      gender      height_cm      weight_kg
## Min.   :21.00   Length:12671   Min.    :125.0   Min.    : 26.30
## 1st Qu.:25.00   Class :character   1st Qu.:162.3   1st Qu.: 57.93
## Median :32.00   Mode  :character   Median :169.0   Median : 67.20
## Mean   :36.76                      Mean   :168.4   Mean    : 67.14
## 3rd Qu.:48.00                      3rd Qu.:174.6   3rd Qu.: 75.00
## Max.   :64.00                      Max.    :193.8   Max.    :138.10
##      body_fat      diastolic      systolic      grip_force
## Min.    : 3.05   Min.    : 49.0   Min.    : 89.0   Min.    : 0.00
## 1st Qu.:17.90   1st Qu.: 71.0   1st Qu.:120.0   1st Qu.:27.50
## Median :22.60   Median : 79.0   Median :130.0   Median :38.00
## Mean    :23.08   Mean    : 78.8   Mean    :130.2   Mean    :37.01
## 3rd Qu.:27.80   3rd Qu.: 86.0   3rd Qu.:141.0   3rd Qu.:45.30
## Max.    :42.65   Max.    :108.0   Max.    :171.0   Max.    :70.50
## sit_and_bend_forward_cm sit_ups_counts broad_jump_cm      class
## Min.    : 0.00           Min.    : 0.00   Min.    : 0.0   Length:12671
## 1st Qu.:11.80           1st Qu.:31.00   1st Qu.:163.0   Class :character
## Median :16.60           Median :42.00   Median :194.0   Mode  :character
## Mean    :16.24           Mean    :40.21   Mean    :190.8
## 3rd Qu.:21.00           3rd Qu.:51.00   3rd Qu.:222.0
## Max.    :34.80           Max.    :80.00   Max.    :303.0

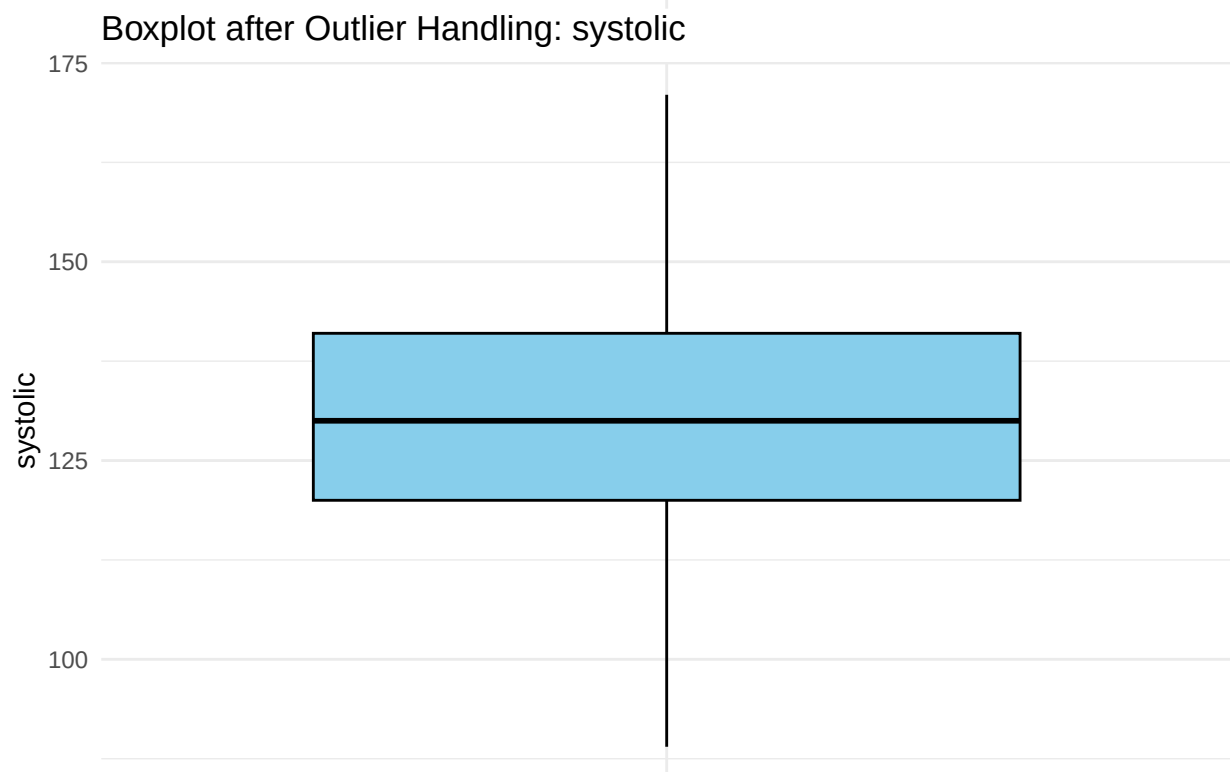
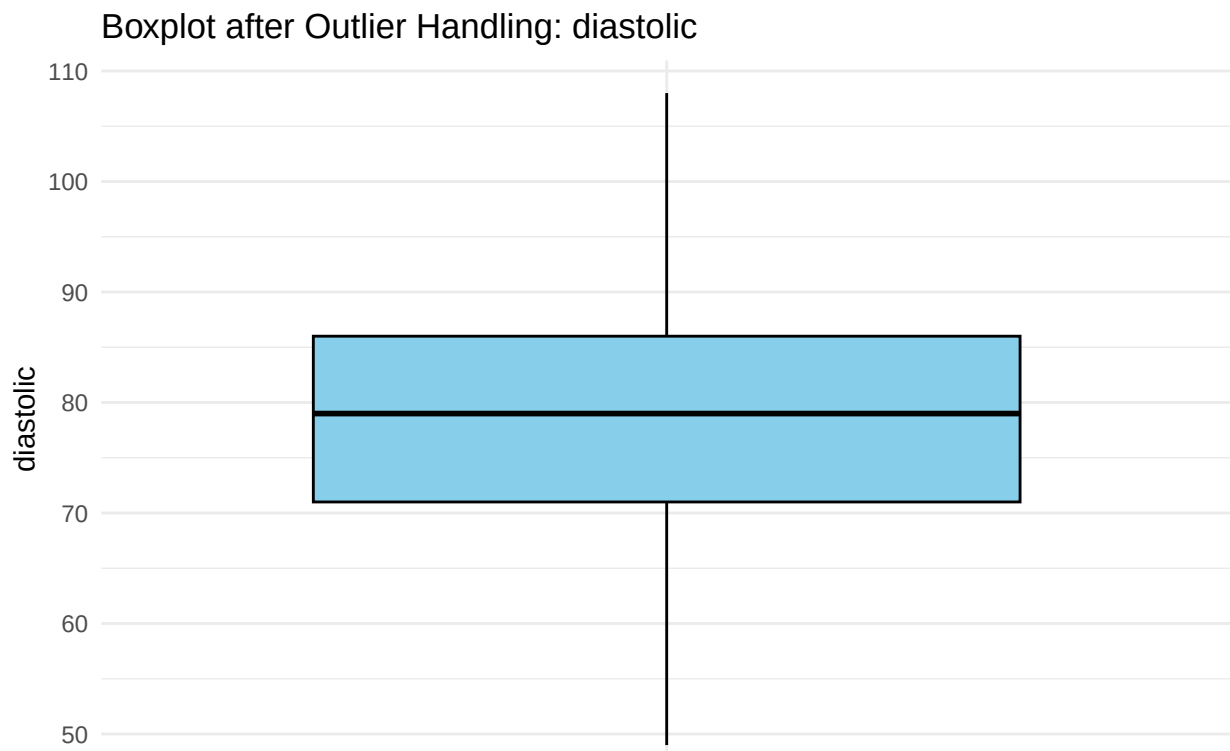
```

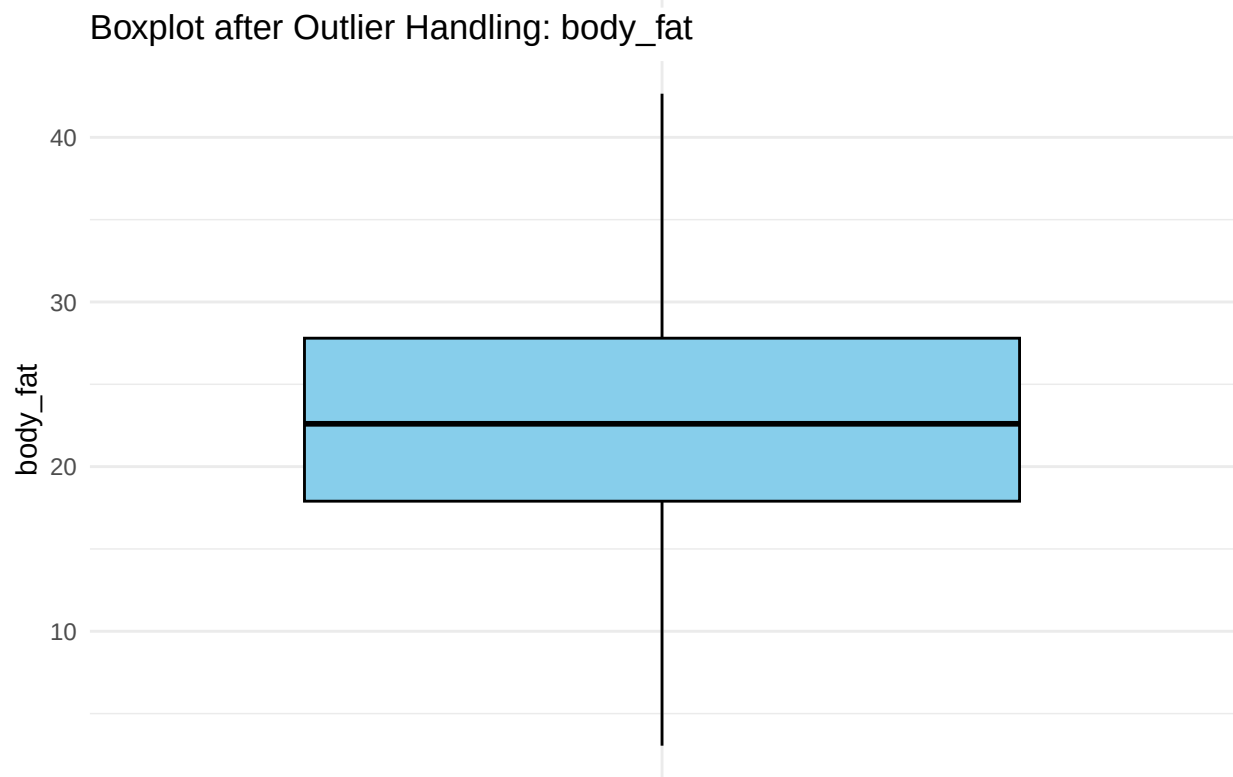
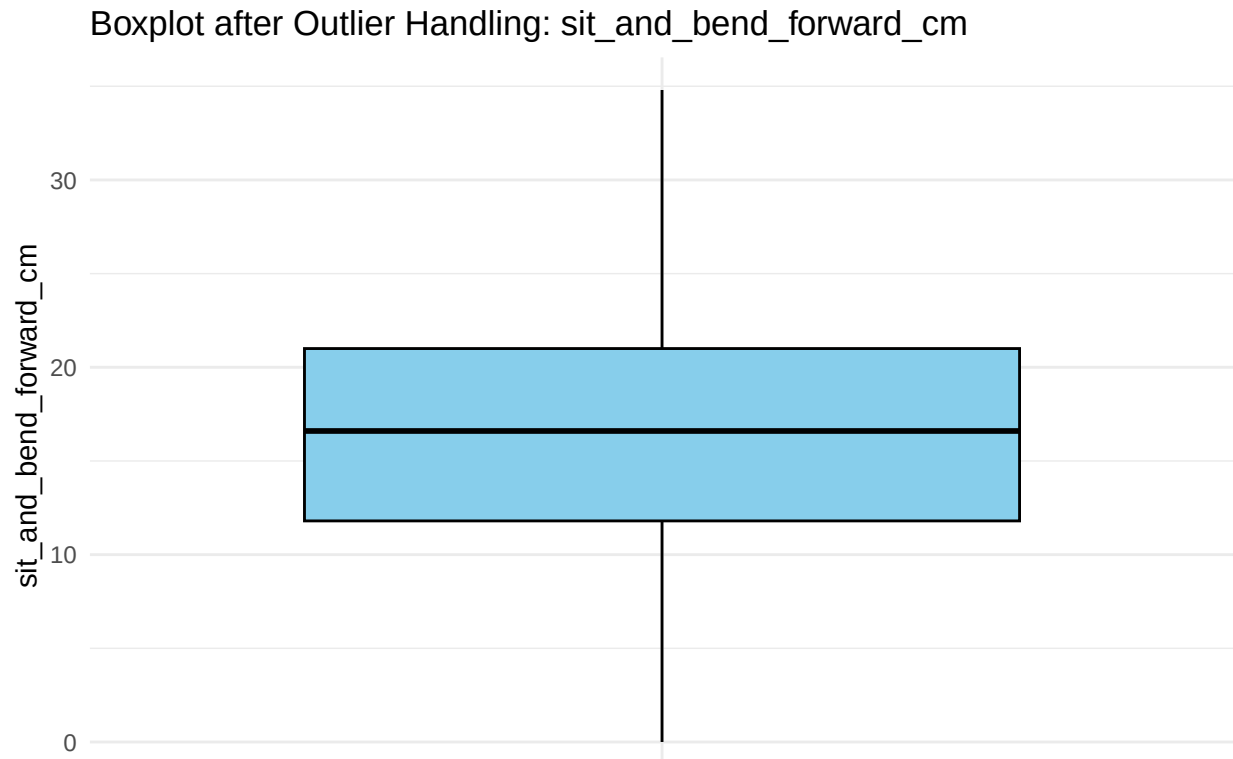
```

for (feature in c("diastolic", "systolic", "sit_and_bend_forward_cm", "body_fat")) {
  print(
    ggplot(data, aes(x = "", y = .data[[feature]])) +
      geom_boxplot(fill = "skyblue", color = "black") +
      labs(title = paste("Boxplot after Outlier Handling:", feature), x = "", y = feature) +
      theme_minimal()
  )
}

```

```
}
```

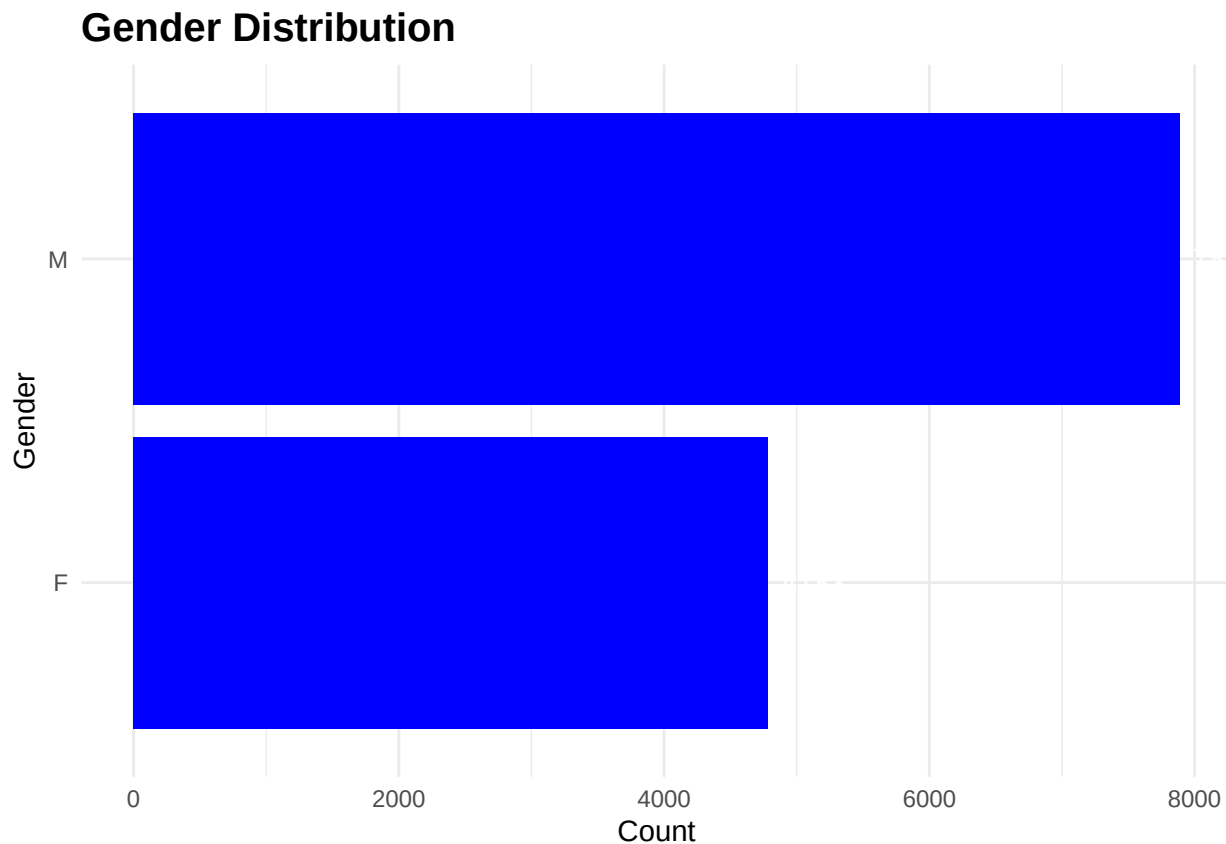




Visualize cleaned data

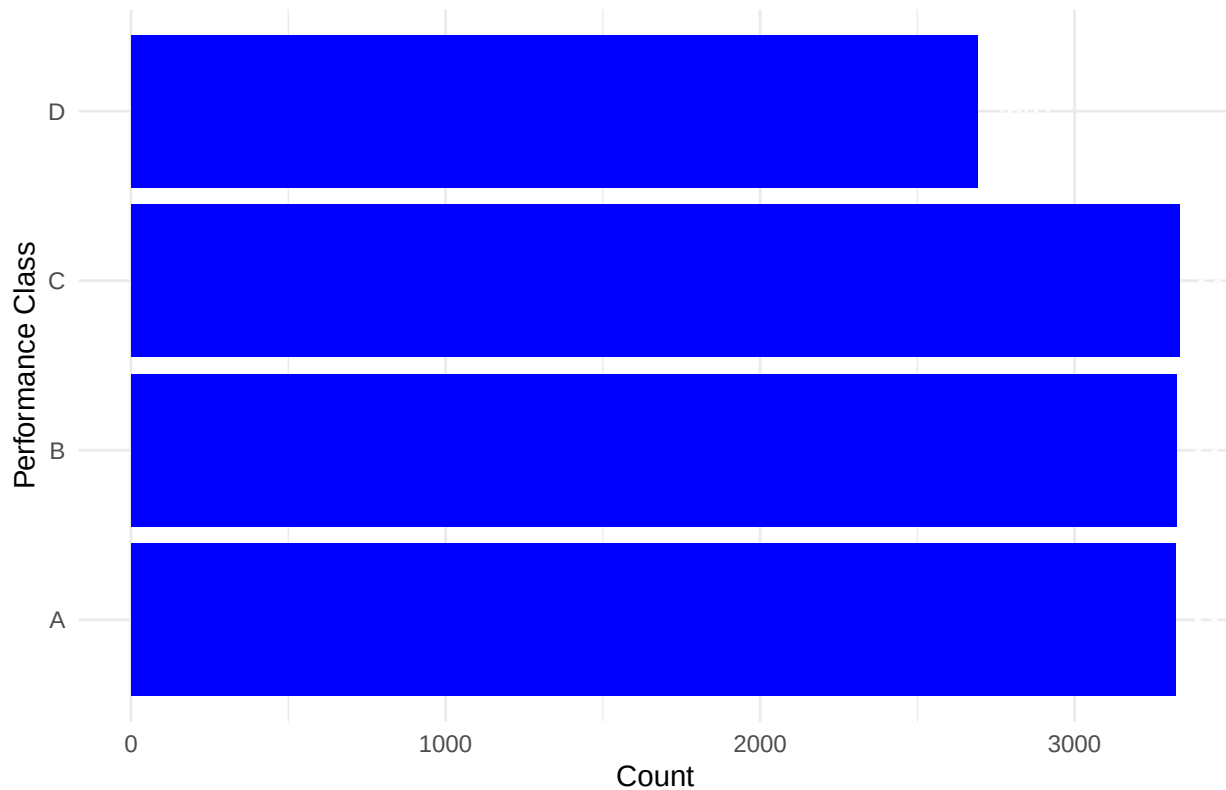
Gender and class distribution

```
ggplot(data, aes(x = gender)) +  
  geom_bar(fill = "blue") +  
  coord_flip() +  
  geom_text(stat = "count", aes(label = after_stat(count)), hjust = -0.2, color = "white") +  
  labs(title = "Gender Distribution", x = "Gender", y = "Count") +  
  theme_minimal() +  
  theme(plot.title = element_text(size = 15, face = "bold"))
```



```
ggplot(data, aes(x = class)) +  
  geom_bar(fill = "blue") +  
  coord_flip() +  
  geom_text(stat = "count", aes(label = after_stat(count)), hjust = -0.2, color = "white") +  
  labs(title = "Class Distribution", x = "Performance Class", y = "Count") +  
  theme_minimal() +  
  theme(plot.title = element_text(size = 15, face = "bold"))
```

Class Distribution



```
table( data$class)
```

```
##  
##      A      B      C      D  
## 3322 3324 3334 2691
```

- After processing, class D is imbalanced compared to other classes.

Frequency and density plot of each numerical variable

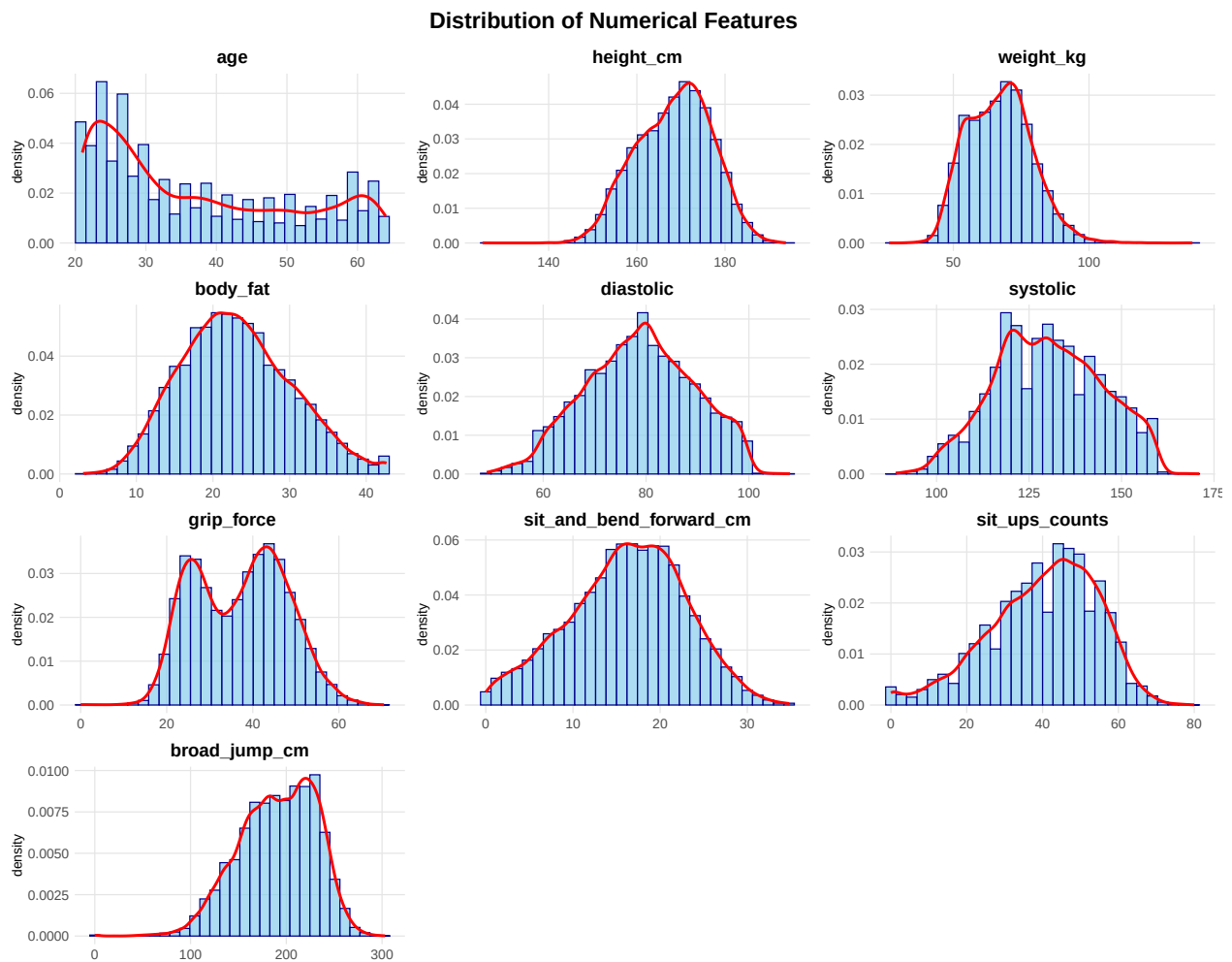
```
hist_plots <- list()  
  
for (var in numerical_features) {  
  p <- ggplot(data, aes(x = .data[[var]])) +  
    geom_histogram(aes(y = after_stat(density)),  
                  fill = "skyblue",  
                  color = "darkblue",  
                  alpha = 0.7,  
                  bins = 30) +  
    geom_density(color = "red", linewidth = 1.2) +  
    labs(title = var) +  
    theme_minimal() +  
    theme(  
      plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
```

```

panel.grid.major = element_line(color = "grey90"),
panel.grid.minor = element_blank(),
axis.title.x = element_blank(),
axis.title.y = element_text(size = 12),
axis.text = element_text(size = 12)
)
hist_plots[[var]] <- p
}

grid.arrange(
  grobs = hist_plots,
  ncol = 3,
  nrow = 4,
  top = textGrob(
    "Distribution of Numerical Features",
    gp = gpar(fontsize = 20, fontface = "bold")
  ),
  padding = unit(2, "line")
)

```

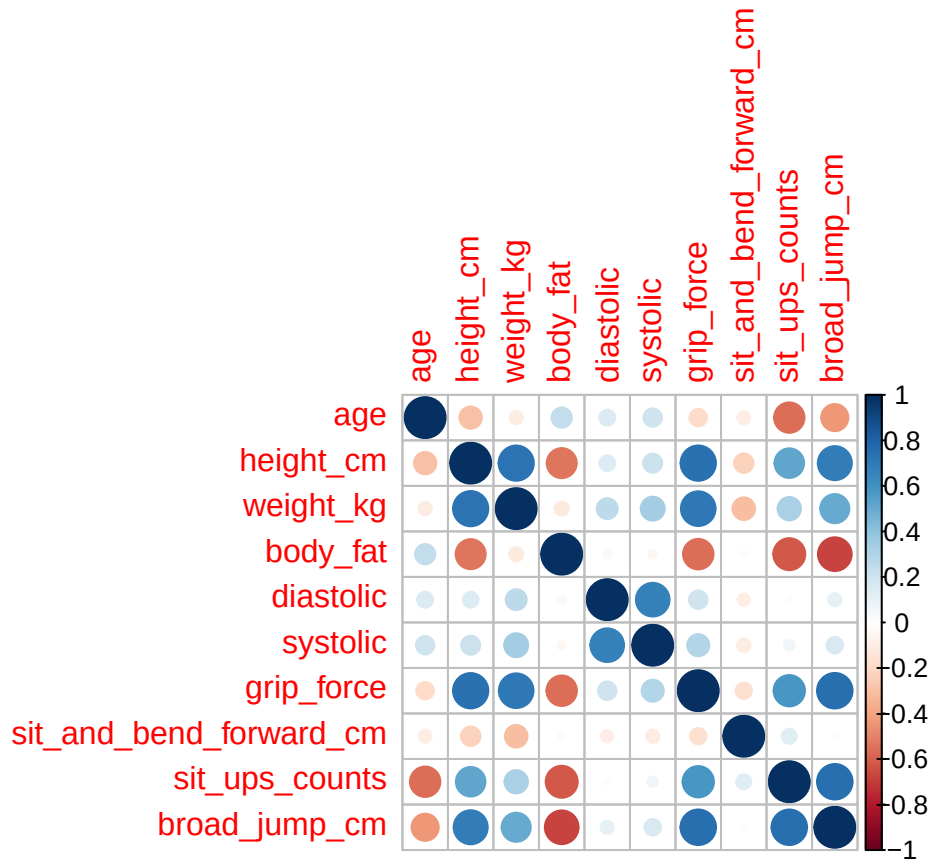


Correlation heatmap

```
numeric_df <- data[, numerical_features]
cor_body <- cor(numeric_df, method = "pearson")
cor_body
```

```
##              age height_cm weight_kg  body_fat
## age          1.00000000 -0.2961313 -0.1055075  0.24448506
## height_cm    -0.29613126  1.0000000  0.7387193 -0.53708051
## weight_kg    -0.10550746  0.7387193  1.0000000 -0.11930143
## body_fat      0.24448506 -0.5370805 -0.1193014  1.00000000
## diastolic     0.15310248  0.1484005  0.2609911  0.03928072
## systolic      0.20363707  0.2156314  0.3419174 -0.04280930
## grip_force   -0.18357680  0.7435728  0.7161117 -0.55349151
## sit_and_bend_forward_cm -0.09979886 -0.2272684 -0.3049417 -0.02801400
## sit_ups_counts -0.55063401  0.5223872  0.3253711 -0.61027811
## broad_jump_cm -0.43906540  0.6900669  0.5061549 -0.67736980
##              diastolic  systolic grip_force
## age          0.15310248  0.20363707 -0.1835768
## height_cm     0.14840053  0.21563143  0.7435728
## weight_kg      0.26099112  0.34191741  0.7161117
## body_fat       0.03928072 -0.04280930 -0.5534915
## diastolic      1.00000000  0.67517168  0.2050871
## systolic       0.67517168  1.00000000  0.2904953
## grip_force     0.20508709  0.29049529  1.0000000
## sit_and_bend_forward_cm -0.09073503 -0.10908142 -0.1610210
## sit_ups_counts  0.02490216  0.06560351  0.5852108
## broad_jump_cm  0.10374325  0.16189360  0.7525447
##              sit_and_bend_forward_cm sit_ups_counts broad_jump_cm
## age          -0.09979886    -0.55063401    -0.43906540
## height_cm     -0.22726842     0.52238718     0.69006691
## weight_kg     -0.30494174     0.32537111     0.50615489
## body_fat      -0.02801400    -0.61027811    -0.67736980
## diastolic     -0.09073503     0.02490216     0.10374325
## systolic      -0.10908142     0.06560351     0.16189360
## grip_force    -0.16102102     0.58521077     0.75254470
## sit_and_bend_forward_cm  1.00000000     0.13459657    -0.02331903
## sit_ups_counts  0.13459657     1.00000000     0.75142710
## broad_jump_cm -0.02331903     0.75142710     1.00000000
```

```
corrplot(cor_body)
```



Perform some tests

Boxplot and violin plot for variables

```
violin_box_list <- list()

for(var in numerical_features) {
  violin_box_list[[var]] <- ggplot(data, aes(x = class, y = .data[[var]], fill = class)) +
    geom_violin(alpha = 0.5) +
    geom_boxplot(width = 0.2, alpha = 0.7) +
    scale_fill_brewer(palette = "Set3") +
    labs(
      title = paste(var),
      x = "Performance Class",
      y = var
    ) +
    theme_minimal() +
    theme(
      plot.title = element_text(size = 14, hjust = 0.5),
      axis.title = element_text(size = 12),
      axis.text = element_text(size = 10),
      panel.grid.major = element_line(color = "grey90"),
      panel.grid.minor = element_blank(),
      legend.position = "none",

```

```

    panel.grid.major.y = element_line(color = "grey90", size = 0.3)
  )
}

```

```

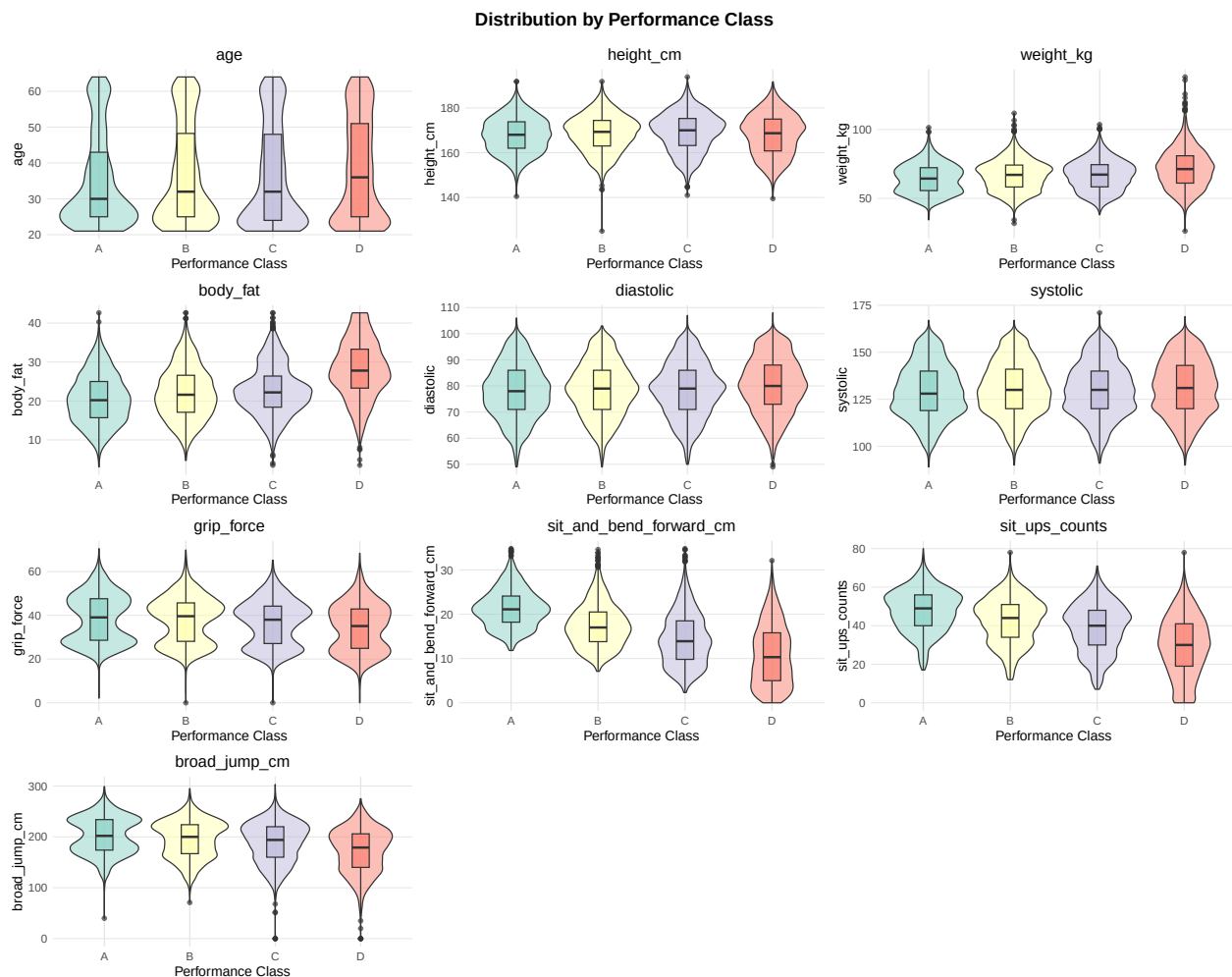
## Warning: The 'size' argument of 'element_line()' is deprecated as of ggplot2 3.4.0.
## i Please use the 'linewidth' argument instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

```

```

grid.arrange(
  grobs = violin_box_list,
  ncol = 3,
  nrow = 4,
  top = textGrob(
    "Distribution by Performance Class",
    gp = gpar(fontsize = 16, fontface = "bold")
  ),
  padding = unit(2, "line")
)

```



Compare variables between performance classes

```
set.seed(54)

anova_results <- list()
plot_list <- list()

for(i in numerical_features) {
  out_aov <- aovp(formula = data[[i]] ~ class, data = data, perm = "Prob")
  anova_results[[i]] <- summary(out_aov)

  p <- ggplot(data, aes(x = class, y = .data[[i]], fill = class)) +
    geom_boxplot() +
    labs(title = paste("Distribution of", i, "by Performance Class"),
         x = "Performance Class",
         y = i) +
    theme_minimal() +
    theme(legend.position = "none") +
    scale_fill_brewer(palette = "Set3")

  plot_list[[i]] <- p

  # ANOVA results
  print(paste("ANOVA Results for", i))
  print(summary(out_aov))
  cat("\n-----\n")
}
```

```
## [1] "Settings: unique SS "
## [1] "ANOVA Results for age"
## Component 1 :
##           Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## class1      3    15250   5083.3 5000 < 2.2e-16 ***
## Residuals 12667 2337658    184.5
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## -----
## [1] "Settings: unique SS "
## [1] "ANOVA Results for height_cm"
## Component 1 :
##           Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## class1      3     3728  1242.55 5000 < 2.2e-16 ***
## Residuals 12667  894578    70.62
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## -----
## [1] "Settings: unique SS "
## [1] "ANOVA Results for weight_kg"
## Component 1 :
```

```

##              Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## class1          3    78306   26102.0 5000 < 2.2e-16 ***
## Residuals    12667  1679470    132.6
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## -----
## [1] "Settings: unique SS "
## [1] "ANOVA Results for body_fat"
## Component 1 :
##              Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## class1          3    92861   30954 5000 < 2.2e-16 ***
## Residuals    12667   557780     44
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## -----
## [1] "Settings: unique SS "
## [1] "ANOVA Results for diastolic"
## Component 1 :
##              Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## class1          3     8327   2775.67 5000 < 2.2e-16 ***
## Residuals    12667  1368336   108.02
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## -----
## [1] "Settings: unique SS "
## [1] "ANOVA Results for systolic"
## Component 1 :
##              Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## class1          3     6651   2216.99 5000 < 2.2e-16 ***
## Residuals    12667  2612349   206.23
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## -----
## [1] "Settings: unique SS "
## [1] "ANOVA Results for grip_force"
## Component 1 :
##              Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## class1          3    30441   10147.0 5000 < 2.2e-16 ***
## Residuals    12667  1417283   111.9
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## -----
## [1] "Settings: unique SS "
## [1] "ANOVA Results for sit_and_bend_forward_cm"

```

```
## Component 1 :
##           Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## class1      3   179828    59943 5000 < 2.2e-16 ***
## Residuals 12667   375874         30
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## -----
## [1] "Settings: unique SS "
## [1] "ANOVA Results for sit_ups_counts"
## Component 1 :
##           Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## class1      3   529594   176531 5000 < 2.2e-16 ***
## Residuals 12667  2020013        159
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## -----
## [1] "Settings: unique SS "
## [1] "ANOVA Results for broad_jump_cm"
## Component 1 :
##           Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## class1      3  1414897   471632 5000 < 2.2e-16 ***
## Residuals 12667 18722617       1478
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## -----

anova_summary <- data.frame(
  Variable = character(),
  F_value = numeric(),
  P_value = numeric(),
  Significant = character(),
  stringsAsFactors = FALSE
)
```

- All variables have very small p-value, indicating that the difference is statistically significant.
- **Analysis by variable group:**
 - **Strength indicators** (grip_force, broad_jump_cm):
 - * Clear difference between classes
 - * Class A and B show outstanding superiority
 - **Endurance indicators** (sit_ups_counts):
 - Clear difference between classes
 - Class A shows outstanding superiority
 - **Body composition indicators** (body_fat, weight_kg):

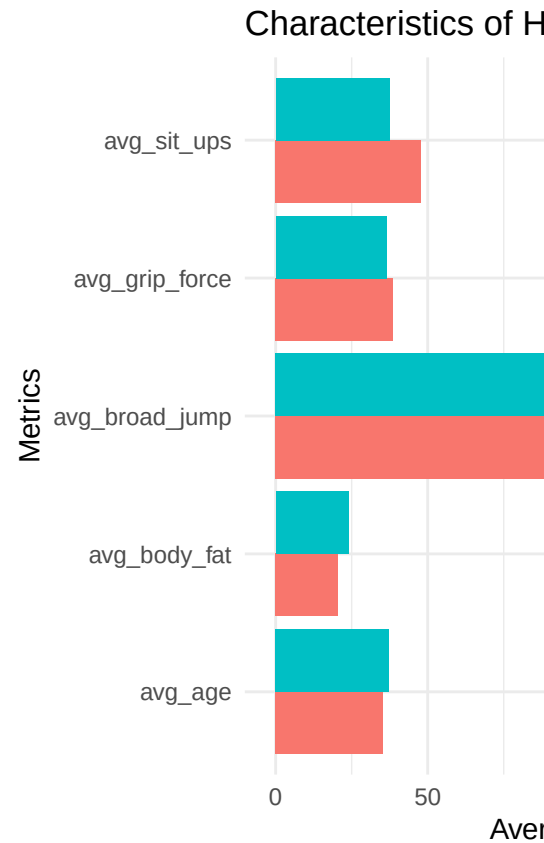
- Clear difference between classes
- Class A shows outstanding superiority

Meaning: - All variables have very small p-value, indicating that the difference is statistically significant. - Strength indicators (grip_force, broad_jump_cm): Clear difference between classes - Class A and B show outstanding superiority - Endurance indicators (sit_ups_counts): Clear difference between classes - Class A shows outstanding superiority - Body composition indicators (body_fat, weight_kg): Clear difference between classes - Class A shows outstanding superiority

```
# Compare indices between Class A and other classes
class_comparison <- data %>%
  mutate(performance_group = ifelse(class == "A", "High Performance", "Others")) %>%
  group_by(performance_group) %>%
  summarise(
    avg_age = mean(age),
    avg_body_fat = mean(body_fat),
    avg_grip_force = mean(grip_force),
    avg_sit_ups = mean(sit_ups_counts),
    avg_broad_jump = mean(broad_jump_cm)
  )

# Visualize the difference
class_comparison_long <- class_comparison %>%
  pivot_longer(cols = starts_with("avg_"),
               names_to = "metric",
               values_to = "value")

ggplot(class_comparison_long,
       aes(x = metric, y = value, fill = performance_group)) +
  geom_bar(stat = "identity", position = "dodge") +
  coord_flip() +
  labs(title = "Characteristics of High Performance Group vs Others",
       x = "Metrics",
       y = "Average Value") +
  theme_minimal()
```



Analyze the characteristics of the high performance group (Class A)

Analyze the impact of gender on performance

```
important_vars <- c("age", "body_fat", "grip_force", "sit_ups_counts", "broad_jump_cm")

# Perform permutation ANOVA for gender x class interaction
cat("Two-way permutation ANOVA (Gender x Class):\n")
```

```
## Two-way permutation ANOVA (Gender x Class):
```

```
for(var in important_vars) {
  formula <- as.formula(paste(var, "~ gender * class"))
  out_aov <- aovp(formula, data = data, perm = "Prob")
  cat(sprintf("\nTwo-way permutation ANOVA for %s:\n", var))
  print(summary(out_aov))
}
```

```
## [1] "Settings: unique SS "
```

```
##
```

```
## Two-way permutation ANOVA for age:
```

```
## Component 1 :
```

```
##           Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## gender1      1   13822   13822.4  5000 < 2.2e-16 ***
## class1       3   25025    8341.6  5000 < 2.2e-16 ***
## gender1:class1 3   36706   12235.4  5000 < 2.2e-16 ***
```

```

## Residuals      12663  2290791    180.9
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## [1] "Settings: unique SS "
##
## Two-way permutation ANOVA for body_fat:
## Component 1 :
##           Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## gender1      1  220348   220348  5000 < 2.2e-16 ***
## class1        3   96500   32167  5000 < 2.2e-16 ***
## gender1:class1 3     384     128  5000 < 2.2e-16 ***
## Residuals    12663  335356     26
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## [1] "Settings: unique SS "
##
## Two-way permutation ANOVA for grip_force:
## Component 1 :
##           Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## gender1      1  953074   953074  5000 < 2.2e-16 ***
## class1        3   45969   15323  5000 < 2.2e-16 ***
## gender1:class1 3     607     202  5000 < 2.2e-16 ***
## Residuals    12663  451887     36
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## [1] "Settings: unique SS "
##
## Two-way permutation ANOVA for sit_ups_counts:
## Component 1 :
##           Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## gender1      1  699735   699735  5000 < 2.2e-16 ***
## class1        3  608187   202729  5000 < 2.2e-16 ***
## gender1:class1 3   26115     8705  5000 < 2.2e-16 ***
## Residuals    12663  1312700    104
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## [1] "Settings: unique SS "
##
## Two-way permutation ANOVA for broad_jump_cm:
## Component 1 :
##           Df R Sum Sq R Mean Sq Iter  Pr(Prob)
## gender1      1 10838491  10838491  5000 < 2.2e-16 ***
## class1        3  1833902   611301  5000 < 2.2e-16 ***
## gender1:class1 3   46437   15479  5000 < 2.2e-16 ***
## Residuals    12663  7869750     621
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

data_long <- data %>%
  pivot_longer(cols = all_of(important_vars),

```

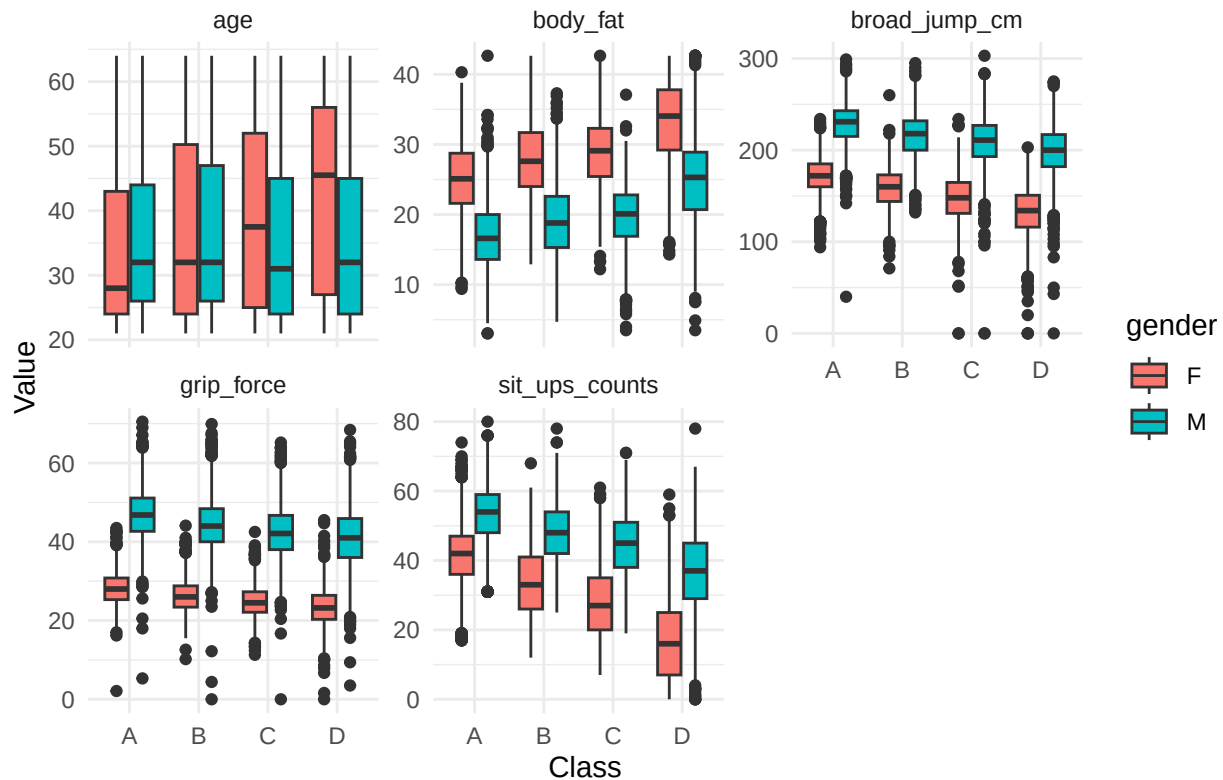
```

names_to = "variable",
values_to = "value")

ggplot(data_long, aes(x = class, y = value, fill = gender)) +
  geom_boxplot() +
  facet_wrap(~variable, scales = "free_y") +
  labs(title = "Distribution of metrics by class and gender",
       x = "Class",
       y = "Value") +
  theme_minimal()

```

Distribution of metrics by class and gender



```

performance_stats <- data %>%
  group_by(gender, class) %>%
  summarise(
    count = n(),
    avg_age = mean(age),
    avg_body_fat = mean(body_fat),
    avg_grip_force = mean(grip_force),
    avg_sit_ups = mean(sit_ups_counts),
    avg_broad_jump = mean(broad_jump_cm)
  ) %>%
  group_by(gender) %>%
  mutate(percentage = count/sum(count) * 100)

```

'summarise()' has grouped output by 'gender'. You can override using the
'.groups' argument.

```
print("\nStatistics by gender and class:")
```

```
## [1] "\nStatistics by gender and class:"
```

```
print(performance_stats)
```

```
## # A tibble: 8 x 9
## # Groups:   gender [2]
##   gender class count avg_age avg_body_fat avg_grip_force avg_sit_ups
##   <chr>   <chr> <int>   <dbl>       <dbl>         <dbl>       <dbl>
## 1 F      A      1475   34.0        25.1          28.2        41.3
## 2 F      B      1176   37.2        27.6          26.3        33.7
## 3 F      C      1106   38.9        28.8          24.8        27.9
## 4 F      D      1026   42.9        33.3          23.4        17.2
## 5 M      A      1847   36.2        16.9          47.0        53.1
## 6 M      B      2148   37.0        19.0          44.3        47.6
## 7 M      C      2228   35.6        19.6          42.5        44.1
## 8 M      D      1665   35.6        24.8          41.1        37.2
## # i 2 more variables: avg_broad_jump <dbl>, percentage <dbl>
```

Note:

1. Role of gender in physical performance:

- Gender is a strong factor affecting all physical indices.
- All p-values $< 2.2e-16$, indicating that the difference between men and women is statistically significant.
- Men generally have an advantage in physical performance, particularly in indices such as grip strength and jump distance.

2. Class differentiation:

- There is a clear difference between classes in all physical indices.
- The gap in physical performance between men and women increases as the class level increases.
- Class A stands out with outstanding physical indices and low body fat percentage.

3. Gender interaction with class:

- The difference between men and women is not uniform across classes.
- The gap in physical performance between men and women increases as the class level increases.
- This suggests that men and women may need different training programs to optimize development.

4. Practical meaning:

- Need to build individual training programs for men and women.

Model

Model evaluation function

```
eval_multi_class <- function(x) {
  cc <- sum(diag(x))
  sc <- sum(x)
  pp <- colSums(x)
  tt <- rowSums(x)

  # Precision and Recall for each class
  prec <- diag(x) / pp
  recall <- diag(x) / tt

  # Macro Precision, Recall and F1
  macro_prec <- mean(prec, na.rm = TRUE)
  macro_recall <- mean(recall, na.rm = TRUE)
  macro_f1 <- 2 * macro_prec * macro_recall / (macro_prec + macro_recall)

  # Accuracy
  acc <- cc / sc

  # Kappa
  kap <- (cc * sc - sum(pp * tt)) / (sc^2 - sum(pp * tt))

  return(list(
    Precision = prec,
    Recall = recall,
    Accuracy = acc,
    Kappa = kap,
    Macro_F1 = macro_f1
  ))
}
```

Data split

```
set.seed(123)
split <- initial_split(data, prop = 0.8, strata = class)
train <- training(split)
test <- testing(split)

cat("Train size: ", dim(train), "\n")
```

```
## Train size: 10135 12
```

```
cat("Test size: ", dim(test), "\n")
```

```
## Test size: 2536 12
```

```
# Check data type
str(train$class)
```

```
## chr [1:10135] "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" "A" ...
```

```
str(test$class)
```

```
## chr [1:2536] "C" "B" "C" "A" "B" "C" "B" "B" "C" "C" "A" "B" "D" "B" "C" ...
```

```
# Convert categorical variables to factor
```

```
train <- train %>%  
  mutate(  
    class = as.factor(class),  
    gender = as.factor(gender)  
  )
```

```
test <- test %>%  
  mutate(  
    class = as.factor(class),  
    gender = as.factor(gender)  
  )  
str(train$class)
```

```
## Factor w/ 4 levels "A","B","C","D": 1 1 1 1 1 1 1 1 1 1 ...
```

```
str(test$class)
```

```
## Factor w/ 4 levels "A","B","C","D": 3 2 3 1 2 3 2 2 3 3 ...
```

Balance data with SMOTE

```
library(themis)  
library(recipes)  
library(tidymodels)  
  
original_distribution <- table(train$class)  
  
recipe_obj <- recipe(class ~ ., data = train) %>%  
  step_smotenc(class)  
  
data_balanced <- prep(recipe_obj) %>%  
  bake(new_data = NULL)  
  
cat("Distribution of class before balancing:\n")
```

```
## Distribution of class before balancing:
```

```
print(original_distribution)
```

```
##  
##      A      B      C      D  
## 2657 2659 2667 2152
```

```
cat("\nDistribution of class after balancing:\n")
```

```
##
```

```
## Distribution of class after balancing:
```

```
print(table(data_balanced$class))
```

```
##
```

```
##      A      B      C      D
```

```
## 2667 2667 2667 2667
```

```
# Visualization
```

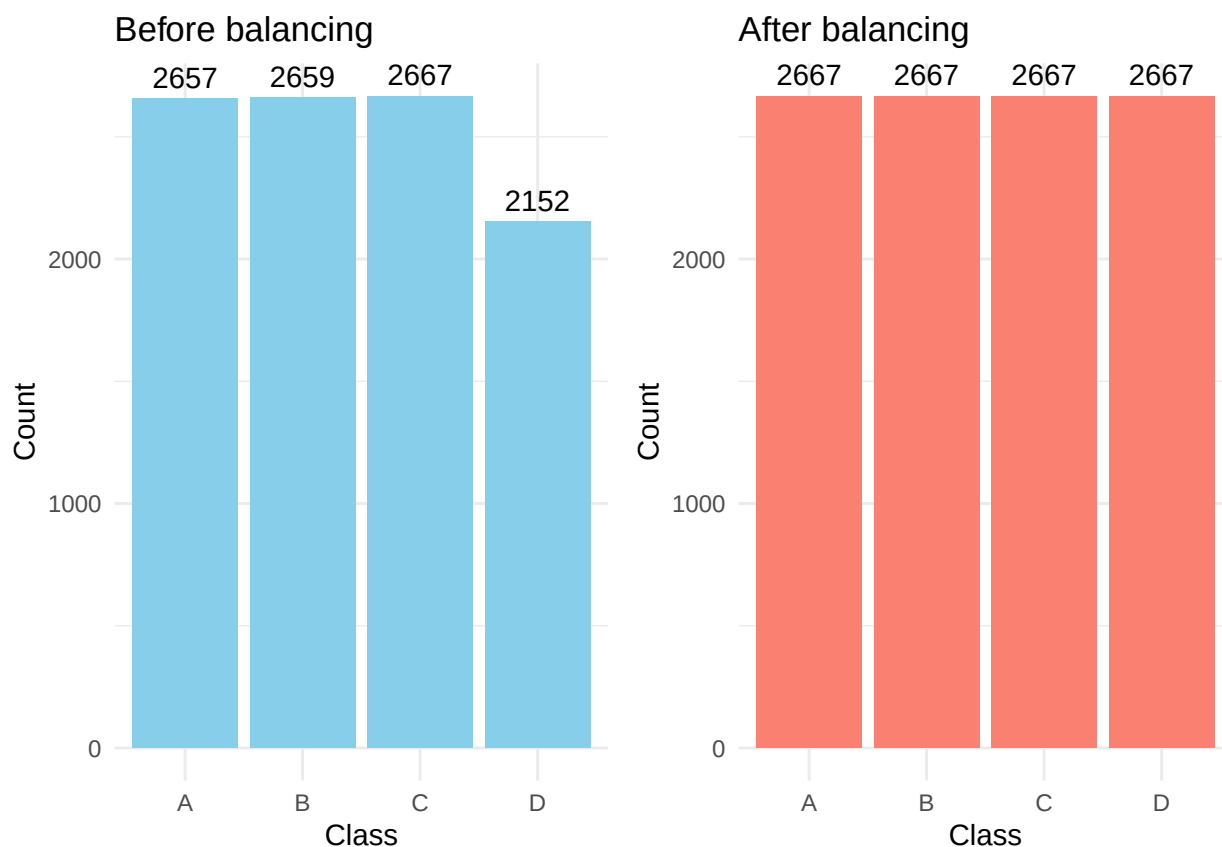
```
p1 <- ggplot(train, aes(x = class)) +  
  geom_bar(fill = "skyblue") +  
  geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.5) +  
  labs(title = "Before balancing",  
        x = "Class",  
        y = "Count") +  
  theme_minimal()
```

```
p2 <- ggplot(data_balanced, aes(x = class)) +  
  geom_bar(fill = "salmon") +  
  geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.5) +  
  labs(title = "After balancing",  
        x = "Class",  
        y = "Count") +  
  theme_minimal()
```

```
# Display both plots
```

```
library(gridExtra)
```

```
grid.arrange(p1, p2, ncol = 2)
```



```
summary(data_balanced)
```

```
##      age      gender  height_cm  weight_kg  body_fat
## Min.   :21.00  F:4019  Min.   :125.0  Min.   : 33.70  Min.   : 3.05
## 1st Qu.:25.00  M:6649  1st Qu.:162.3  1st Qu.: 58.10  1st Qu.:18.00
## Median :32.00          Median :169.0  Median : 67.50  Median :22.90
## Mean   :36.93          Mean   :168.4  Mean   : 67.36  Mean   :23.31
## 3rd Qu.:48.00          3rd Qu.:174.7  3rd Qu.: 75.20  3rd Qu.:28.20
## Max.   :64.00          Max.   :193.8  Max.   :138.10  Max.   :42.65
## diastolic      systolic    grip_force  sit_and_bend_forward_cm
## Min.   : 49.00  Min.   : 89.0  Min.   : 0.00  Min.   : 0.00
## 1st Qu.: 72.00  1st Qu.:120.0  1st Qu.:27.30  1st Qu.:11.40
## Median : 79.00  Median :130.0  Median :37.92  Median :16.30
## Mean   : 78.94  Mean   :130.4  Mean   :36.95  Mean   :15.96
## 3rd Qu.: 86.17  3rd Qu.:141.0  3rd Qu.:45.30  3rd Qu.:20.80
## Max.   :107.00  Max.   :171.0  Max.   :70.50  Max.   :34.80
## sit_ups_counts  broad_jump_cm  class
## Min.   : 0.00  Min.   : 0  A:2667
## 1st Qu.:30.00  1st Qu.:162 B:2667
## Median :41.00  Median :193 C:2667
## Mean   :39.77  Mean   :190 D:2667
## 3rd Qu.:51.00  3rd Qu.:222
## Max.   :80.00  Max.   :303
```

```
data <- data_balanced
```

Multinomial Logistic Regression

```
multinom_model <- multinom(class ~ ., data = train, maxit = 100)
```

Multinomial Logistic Regression without feature transformation

```
## # weights: 52 (36 variable)
## initial value 14050.093350
## iter 10 value 10842.868721
## iter 20 value 10683.064620
## iter 30 value 10560.516653
## iter 40 value 9076.338898
## final value 9075.252033
## converged
```

```
pred <- predict(multinom_model, newdata = test, type = "class")
head(pred)
```

```
## [1] A D C B C C
## Levels: A B C D
```

```
# confusion matrix
multinom_conf_mat <- tibble(
  truth = test$class,
  prediction = pred
) %>%
  conf_mat(truth = truth, estimate = prediction)

# Evaluate model
metrics_multinom <- eval_multi_class(multinom_conf_mat$table)
print(metrics_multinom)
```

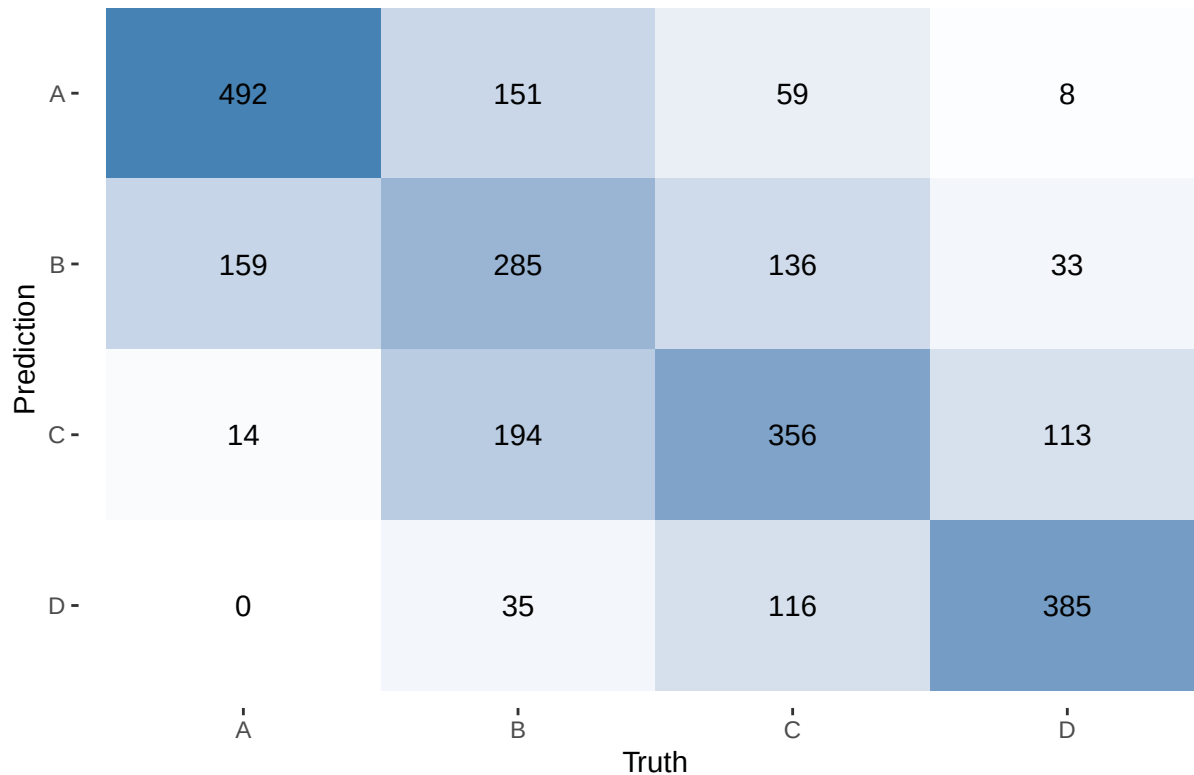
```
## $Precision
##      A      B      C      D
## 0.7398496 0.4285714 0.5337331 0.7142857
##
## $Recall
##      A      B      C      D
## 0.6929577 0.4649266 0.5258493 0.7182836
##
## $Accuracy
## [1] 0.5985804
##
## $Kappa
## [1] 0.4633906
##
## $Macro_F1
## [1] 0.6023017
```

```
# Visualization
autoplot(multinom_conf_mat, type = "heatmap") +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(title = "Confusion Matrix - Multinomial Logistic Regression")
```

```
## Scale for fill is already present.
```

```
## Adding another scale for fill, which will replace the existing scale.
```

Confusion Matrix – Multinomial Logistic Regression



```
multinom_model_simplified <- multinom(
  class ~ gender + age + diastolic + systolic + height_cm +
    I(log(grip_force + 1)) +
    I(log(sit_ups_counts + 1)) +
    I(log(broad_jump_cm + 1)) +
    I(weight_kg * (body_fat/100)) +
    sit_and_bend_forward_cm,
  data = train,
  maxit = 1000
)
```

Train Multinomial Logistic Regression model with interaction, log, and square transformations

```
## # weights: 48 (33 variable)
```

```
## initial value 14050.093350
## iter 10 value 11717.196923
## iter 20 value 11261.867575
## iter 30 value 9839.718698
## iter 40 value 8980.928322
## final value 8979.412863
## converged
```

```
pred_simplified <- predict(multinom_model_simplified, newdata = test, type = "class")
head(pred_simplified)
```

```
## [1] A D D B C B
## Levels: A B C D
```

```
# Confusion Matrix
multinom_conf_mat_simplified <- tibble(
  truth = test$class,
  prediction = pred_simplified
) %>%
  conf_mat(truth = truth, estimate = prediction)

print(multinom_conf_mat_simplified)
```

```
##           Truth
## Prediction  A   B   C   D
##           A 494 154  62  11
##           B 159 297 135  33
##           C  11 187 373 105
##           D   1  27  97 390
```

```
# Evaluate model
metrics_simplified <- eval_multi_class(multinom_conf_mat_simplified$table)
print(metrics_simplified)
```

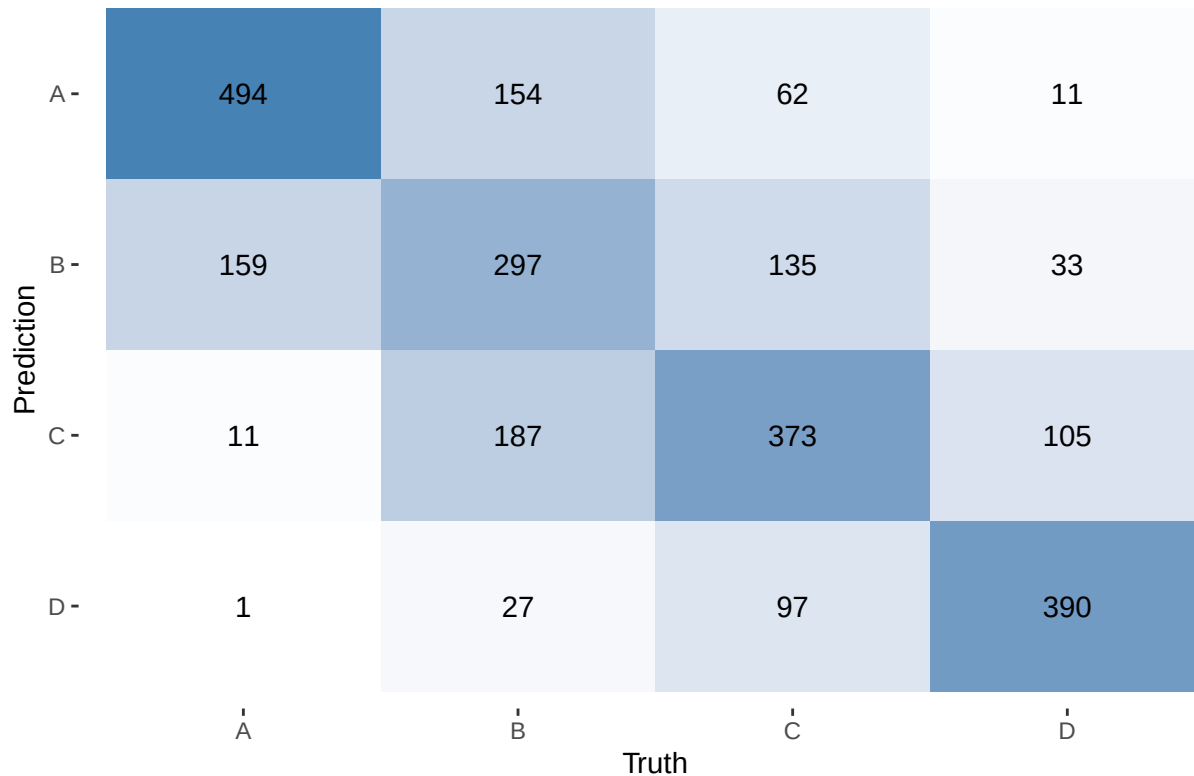
```
## $Precision
##           A           B           C           D
## 0.7428571 0.4466165 0.5592204 0.7235622
##
## $Recall
##           A           B           C           D
## 0.6851595 0.4759615 0.5517751 0.7572816
##
## $Accuracy
## [1] 0.612776
##
## $Kappa
## [1] 0.4820824
##
## $Macro_F1
## [1] 0.6178041
```

```
# Visualization
autoplot(multinom_conf_mat_simplified, type = "heatmap") +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(title = "Confusion Matrix - Simplified Multinomial Model")
```

```
## Scale for fill is already present.
```

```
## Adding another scale for fill, which will replace the existing scale.
```

Confusion Matrix – Simplified Multinomial Model



Random Forest

```
library(randomForest)
rf_model <- randomForest(class ~ ., data = train)
pred_rf <- predict(rf_model, newdata = test, type = "class")
head(pred_rf)
```

```
## 1 2 3 4 5 6
## A D C B A B
## Levels: A B C D
```

```
# Confusion Matrix
rf_conf_mat <- tibble(
  truth = test$class,
  prediction = pred_rf
```

```

) %>%
  conf_mat(truth = truth, estimate = prediction)

print(rf_conf_mat)

```

```

##           Truth
## Prediction  A   B   C   D
##           A 571 149  62   9
##           B  83 399 129  28
##           C   8  85 441  90
##           D   3  32  35 412

```

```

# Evaluate model
metrics_rf <- eval_multi_class(rf_conf_mat$table)
print(metrics_rf)

```

```

## $Precision
##           A           B           C           D
## 0.8586466 0.6000000 0.6611694 0.7643785
##
## $Recall
##           A           B           C           D
## 0.7218710 0.6244131 0.7067308 0.8547718
##
## $Accuracy
## [1] 0.7188486
##
## $Kappa
## [1] 0.6236386
##
## $Macro_F1
## [1] 0.7239856

```

```

# Visualization
autoplot(rf_conf_mat, type = "heatmap") +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(title = "Confusion Matrix - Random Forest")

```

```

## Scale for fill is already present.
## Adding another scale for fill, which will replace the existing scale.

```

Confusion Matrix – Random Forest

Prediction	A -	571	149	62	9
	B -	83	399	129	28
	C -	8	85	441	90
	D -	3	32	35	412
		A	B	C	D
		Truth			

Training XGBoost

```
library(xgboost)

# Prepare data
# Encoding class (A=0, B=1, C=2, D=3)
train$class_num <- as.numeric(factor(train$class, levels = c("A", "B", "C", "D"))) - 1
test$class_num <- as.numeric(factor(test$class, levels = c("A", "B", "C", "D"))) - 1

# Encoding gender (Female=0, Male=1)
train$gender_num <- ifelse(train$gender == "M", 1, 0)
test$gender_num <- ifelse(test$gender == "M", 1, 0)

# Choose numeric features
feature_cols <- c("age", "height_cm", "weight_kg", "body_fat", "diastolic",
                  "systolic", "grip_force", "sit_and_bend_forward_cm",
                  "sit_ups_counts", "broad_jump_cm", "gender_num")

# Convert data to matrix
train_matrix <- as.matrix(train[, feature_cols])
test_matrix <- as.matrix(test[, feature_cols])

# Train model
xgb_model <- xgboost(data = train_matrix,
                    label = train$class_num,
```

```
nrounds = 100,  
objective = "multi:softmax",  
num_class = 4,  
eta = 0.3,  
max_depth = 6)
```

```
## [1] train-mlogloss:1.180097  
## [2] train-mlogloss:1.050373  
## [3] train-mlogloss:0.960655  
## [4] train-mlogloss:0.893058  
## [5] train-mlogloss:0.838748  
## [6] train-mlogloss:0.797137  
## [7] train-mlogloss:0.762834  
## [8] train-mlogloss:0.735008  
## [9] train-mlogloss:0.707137  
## [10] train-mlogloss:0.682477  
## [11] train-mlogloss:0.663257  
## [12] train-mlogloss:0.646681  
## [13] train-mlogloss:0.630795  
## [14] train-mlogloss:0.616852  
## [15] train-mlogloss:0.602799  
## [16] train-mlogloss:0.590784  
## [17] train-mlogloss:0.581445  
## [18] train-mlogloss:0.571439  
## [19] train-mlogloss:0.560099  
## [20] train-mlogloss:0.551533  
## [21] train-mlogloss:0.541841  
## [22] train-mlogloss:0.532235  
## [23] train-mlogloss:0.526476  
## [24] train-mlogloss:0.518476  
## [25] train-mlogloss:0.510508  
## [26] train-mlogloss:0.502729  
## [27] train-mlogloss:0.494787  
## [28] train-mlogloss:0.489512  
## [29] train-mlogloss:0.483469  
## [30] train-mlogloss:0.477470  
## [31] train-mlogloss:0.472606  
## [32] train-mlogloss:0.467679  
## [33] train-mlogloss:0.461243  
## [34] train-mlogloss:0.454979  
## [35] train-mlogloss:0.449156  
## [36] train-mlogloss:0.443980  
## [37] train-mlogloss:0.439392  
## [38] train-mlogloss:0.433307  
## [39] train-mlogloss:0.428895  
## [40] train-mlogloss:0.426746  
## [41] train-mlogloss:0.420720  
## [42] train-mlogloss:0.416580  
## [43] train-mlogloss:0.413379  
## [44] train-mlogloss:0.408579  
## [45] train-mlogloss:0.405162  
## [46] train-mlogloss:0.401218  
## [47] train-mlogloss:0.398199
```

```
## [48] train-mlogloss:0.391725
## [49] train-mlogloss:0.389158
## [50] train-mlogloss:0.385479
## [51] train-mlogloss:0.383262
## [52] train-mlogloss:0.380937
## [53] train-mlogloss:0.378785
## [54] train-mlogloss:0.373916
## [55] train-mlogloss:0.370195
## [56] train-mlogloss:0.366332
## [57] train-mlogloss:0.362132
## [58] train-mlogloss:0.358084
## [59] train-mlogloss:0.353923
## [60] train-mlogloss:0.350780
## [61] train-mlogloss:0.346907
## [62] train-mlogloss:0.344370
## [63] train-mlogloss:0.340283
## [64] train-mlogloss:0.336517
## [65] train-mlogloss:0.334759
## [66] train-mlogloss:0.332108
## [67] train-mlogloss:0.328713
## [68] train-mlogloss:0.324195
## [69] train-mlogloss:0.318924
## [70] train-mlogloss:0.317189
## [71] train-mlogloss:0.313194
## [72] train-mlogloss:0.310563
## [73] train-mlogloss:0.305913
## [74] train-mlogloss:0.302534
## [75] train-mlogloss:0.300988
## [76] train-mlogloss:0.299529
## [77] train-mlogloss:0.297714
## [78] train-mlogloss:0.295496
## [79] train-mlogloss:0.292403
## [80] train-mlogloss:0.289447
## [81] train-mlogloss:0.285598
## [82] train-mlogloss:0.284283
## [83] train-mlogloss:0.281819
## [84] train-mlogloss:0.278735
## [85] train-mlogloss:0.274674
## [86] train-mlogloss:0.272076
## [87] train-mlogloss:0.270351
## [88] train-mlogloss:0.267642
## [89] train-mlogloss:0.264806
## [90] train-mlogloss:0.261052
## [91] train-mlogloss:0.258569
## [92] train-mlogloss:0.255174
## [93] train-mlogloss:0.252755
## [94] train-mlogloss:0.250070
## [95] train-mlogloss:0.246933
## [96] train-mlogloss:0.245049
## [97] train-mlogloss:0.242221
## [98] train-mlogloss:0.240681
## [99] train-mlogloss:0.237940
## [100] train-mlogloss:0.235987
```

```
# Predict
pred_xgb_num <- predict(xgb_model, test_matrix)
# Convert prediction to factor with correct order
pred_xgb <- factor(pred_xgb_num, levels = 0:3, labels = c("A", "B", "C", "D"))
```

```
# Confusion Matrix
xgb_conf_mat <- tibble(
  truth = test$class,
  prediction = factor(pred_xgb, levels = levels(test$class))
) %>%
  conf_mat(truth = truth, estimate = prediction)

print(xgb_conf_mat)
```

```
##           Truth
## Prediction  A   B   C   D
##           A 589 130  57   8
##           B  68 408 133  40
##           C   8 100 450  77
##           D   0  27  27 414
```

```
# Evaluate model
metrics_xgb <- eval_multi_class(xgb_conf_mat$table)
print(metrics_xgb)
```

```
## $Precision
##           A           B           C           D
## 0.8857143 0.6135338 0.6746627 0.7680891
##
## $Recall
##           A           B           C           D
## 0.7512755 0.6286595 0.7086614 0.8846154
##
## $Accuracy
## [1] 0.7338328
##
## $Kappa
## [1] 0.6435647
##
## $Macro_F1
## [1] 0.7393809
```

```
# Visualization
autoplot(xgb_conf_mat, type = "heatmap") +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(title = "Confusion Matrix - XGBoost")
```

```
## Scale for fill is already present.
## Adding another scale for fill, which will replace the existing scale.
```

Confusion Matrix – XGBoost

Prediction	A -	589	130	57	8
	B -	68	408	133	40
	C -	8	100	450	77
	D -	0	27	27	414
		A	B	C	D
		Truth			

Model comparison

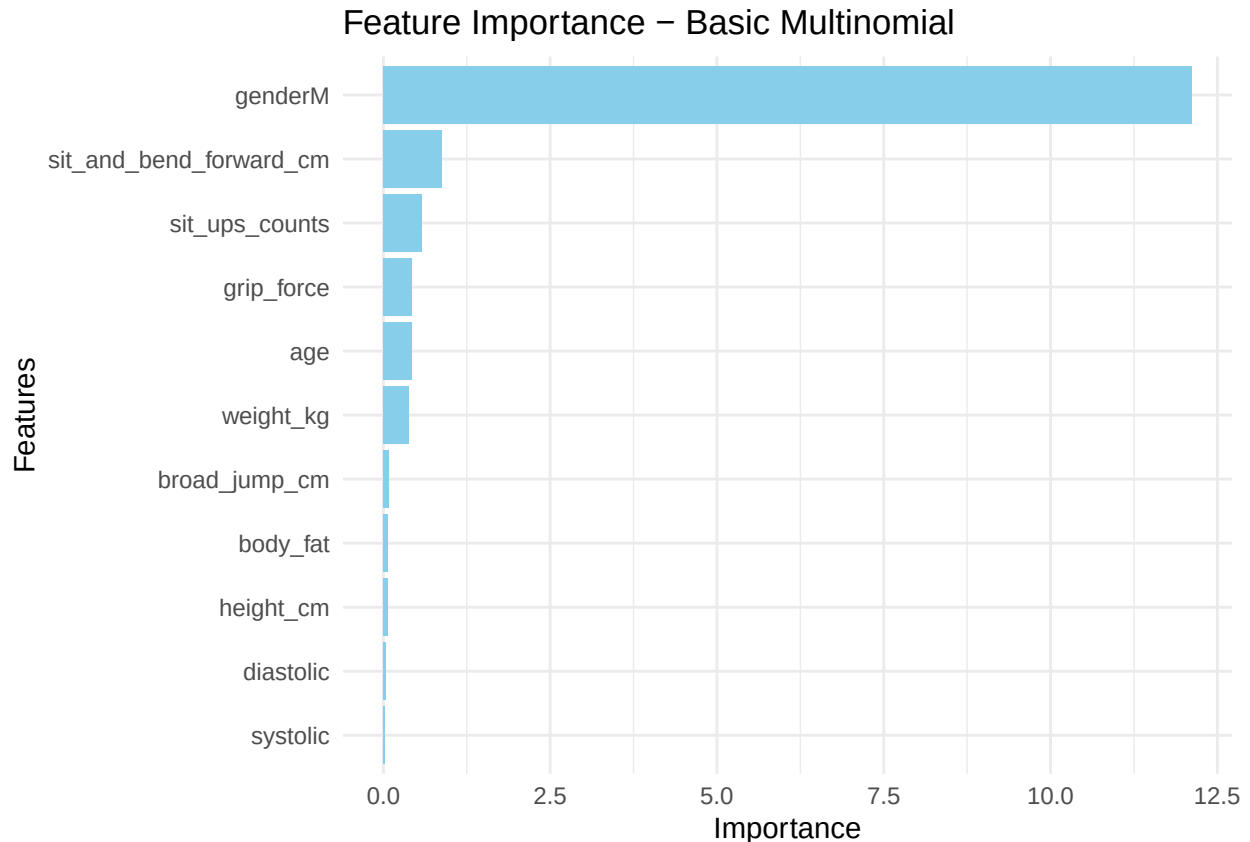
```
comparison <- data.frame(
  Model = c("Basic Multinomial", "Simplified Multinomial", "Random Forest", "XGBoost"),
  Accuracy = c(metrics_multinom$Accuracy, metrics_simplified$Accuracy,
    metrics_rf$Accuracy, metrics_xgb$Accuracy),
  Kappa = c(metrics_multinom$Kappa, metrics_simplified$Kappa,
    metrics_rf$Kappa, metrics_xgb$Kappa),
  Macro_F1 = c(metrics_multinom$Macro_F1, metrics_simplified$Macro_F1,
    metrics_rf$Macro_F1, metrics_xgb$Macro_F1)
)
print(comparison)
```

```
##           Model Accuracy      Kappa Macro_F1
## 1 Basic Multinomial 0.5985804 0.4633906 0.6023017
## 2 Simplified Multinomial 0.6127760 0.4820824 0.6178041
## 3 Random Forest 0.7188486 0.6236386 0.7239856
## 4 XGBoost 0.7338328 0.6435647 0.7393809
```

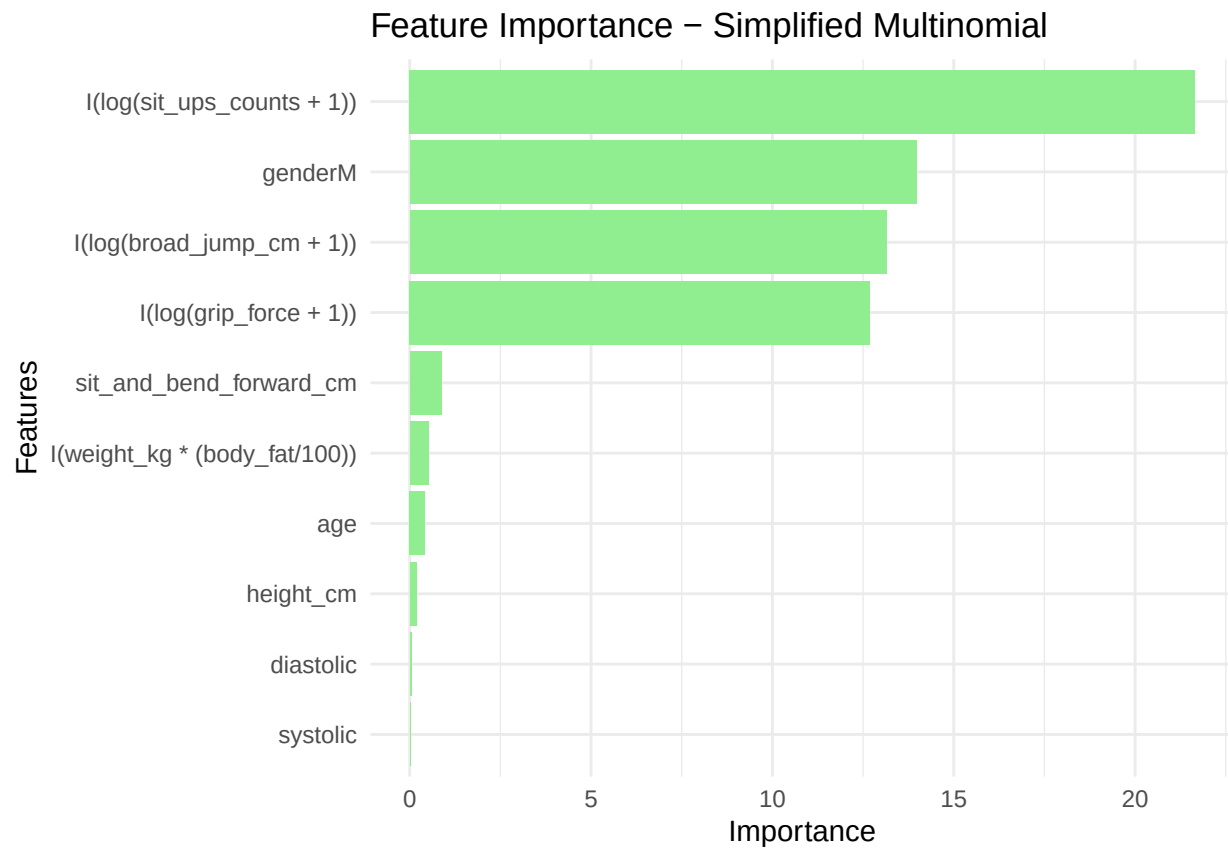
Feature importance

```
# 1.Multinomial
basic_importance <- varImp(multinom_model)
```

```
ggplot(data = data.frame(
  Feature = rownames(basic_importance),
  Importance = basic_importance[,1]
)) +
  geom_bar(aes(x = reorder(Feature, Importance), y = Importance), stat = "identity", fill = "skyblue") +
  coord_flip() +
  labs(title = "Feature Importance - Basic Multinomial",
       x = "Features", y = "Importance") +
  theme_minimal()
```

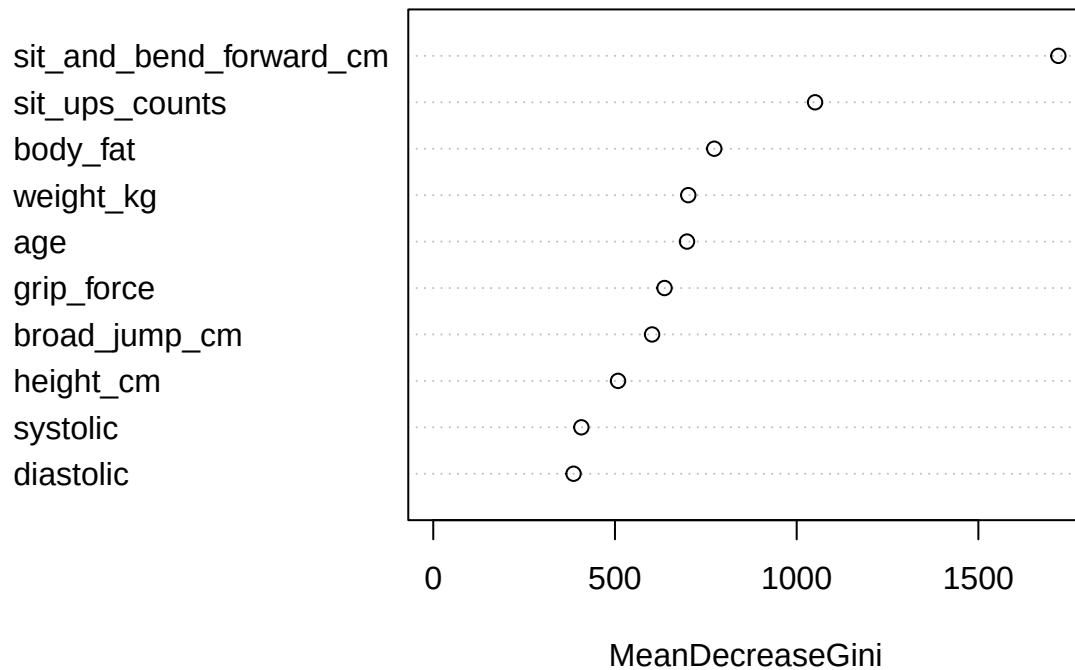


```
# 2. Simplified Multinomial
simplified_importance <- varImp(multinom_model_simplified)
ggplot(data = data.frame(
  Feature = rownames(simplified_importance),
  Importance = simplified_importance[,1]
)) +
  geom_bar(aes(x = reorder(Feature, Importance), y = Importance), stat = "identity", fill = "lightgreen") +
  coord_flip() +
  labs(title = "Feature Importance - Simplified Multinomial",
       x = "Features", y = "Importance") +
  theme_minimal()
```



```
# 3. Random Forest
varImpPlot(rf_model,
  sort = TRUE,
  main = "Feature Importance - Random Forest",
  n.var = min(10, ncol(train)))
```

Feature Importance – Random Forest



```
# 4. XGBoost
importance_matrix <- xgb.importance(
  feature_names = feature_cols,
  model = xgb_model
)
xgb.plot.importance(importance_matrix, top_n = 10,
  main = "Feature Importance - XGBoost")
```

Feature Importance – XGBoost

